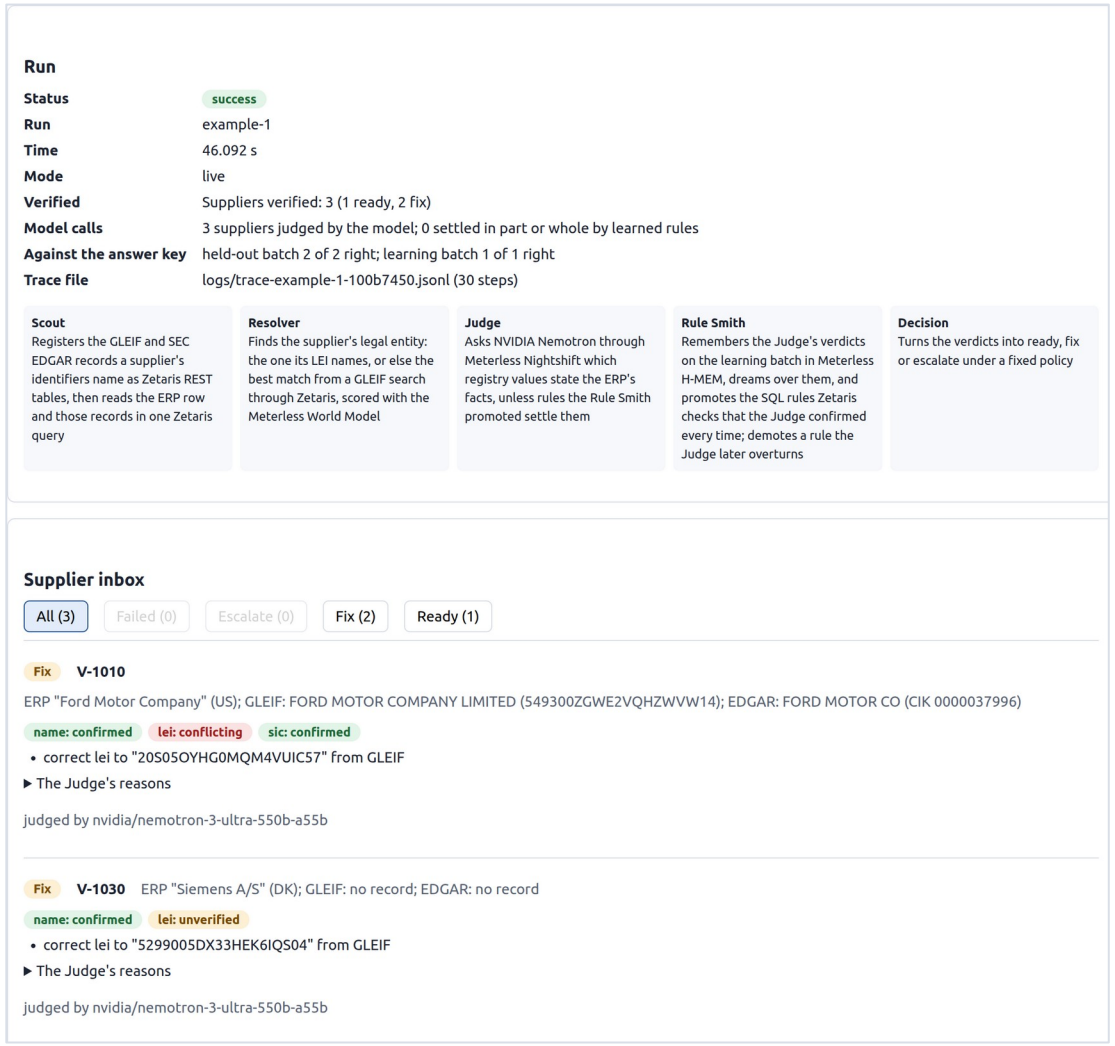
Provenance

Supplier-verification agents that turn their own proven judgments into SQL rules Zetaris enforces

Track 1, Solving Fragmented Intelligence · Open Agent Hackathon 2026

Zetaris | Meterless | NVIDIA Nemotron | Built in Cursor

github.com/Vitalisn4/provenance



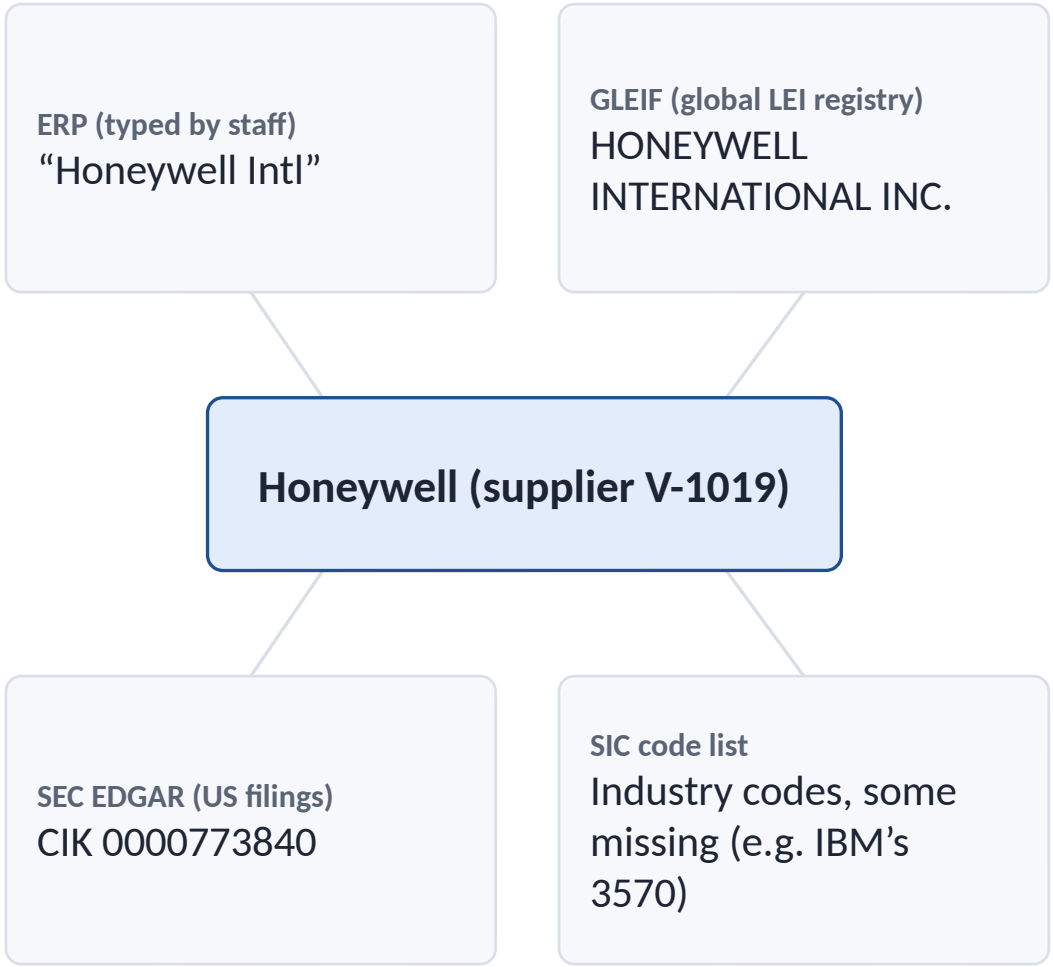
One supplier, four sources, four answers

The user: a supplier onboarding analyst in procurement

The job: confirm each supplier's legal name, LEI (Legal Entity Identifier), registration status and industry code before it enters the ERP (the company's business system, such as SAP)

The pain: the facts are spread across sources that disagree, checked by hand for every supplier

The risk: a language model on its own invents agreement that isn't there



From scattered records to one decision per supplier

Input

- ERP vendor IDs
- Typed-in new suppliers
- Nothing: checks the whole vendor master

Agents

5 agents

Five agents check every fact against the registries through Zetaris

Output, per supplier

Each fact: legal name, LEI, industry code

confirmed | conflicting | unverified

One decision

ready | fix | escalate

Fix means a concrete action: correct field X to value Y from source Z, or request a document

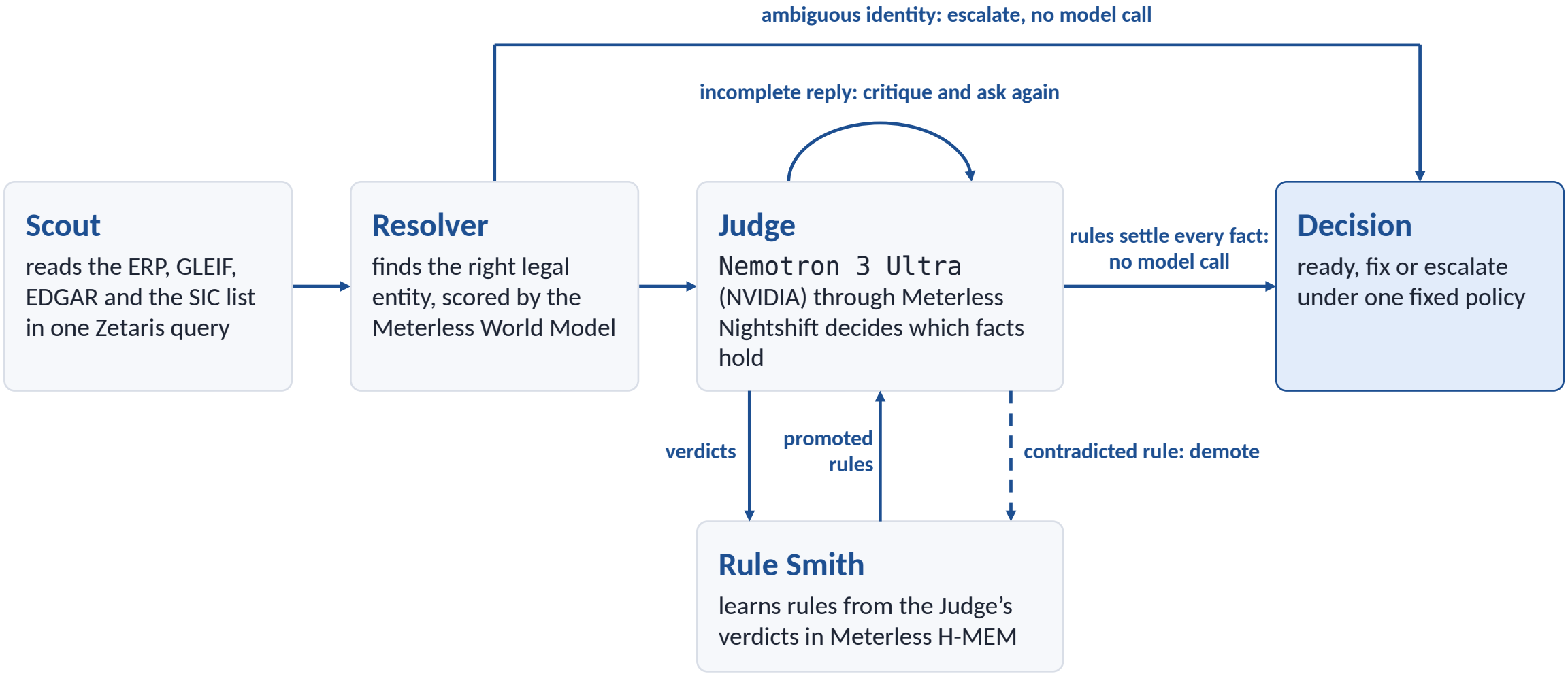
The agent recommends; the analyst decides.

Real problems found in a 31-supplier ERP

Supplier	The ERP says	Provenance finds	Decision
Siemens AG (Germany)	its LEI	that LEI belongs to a same-name company in another country	fix
Honeywell Intl	its name	the legal name is HONEYWELL INTERNATIONAL INC.	fix
IBM	industry code 3570	EDGAR agrees, but the SIC list has no code 3570	escalate
Ford Motor Company A/S	its LEI	the registration has LAPSED	escalate
Ericsson (Sweden)	a name, no identifiers	two registered companies match; neither can be chosen	escalate
Small suppliers in Cameroon, Ghana, Kenya	names only	no registry covers them	fix: request a document

Scored against a hand-labelled answer key: 18/18 on the learning batch, 13/13 on the held-out batch (live run).

Five agents that hand off, challenge and learn



Judgments become SQL that Zetaris runs

1. **Learn:** the Judge verifies a learning batch of 18 suppliers
2. **Remember:** each verdict becomes an H-MEM memory; H-MEM “dreams” over them and proposes patterns
3. **Prove:** each candidate rule is back-tested against every verdict; promoted only if the Judge never contradicted it
4. **Apply:** Zetaris evaluates the promoted rules inside the agents’ own queries on the 13 held-out suppliers

Rules from the live run

Rule	Result
`name_is_gleif_legal_name`	promoted
`lei_is_claimed_record`	promoted
`sic_is_listed_edgar_code`	promoted
`sic_missing_from_list`	promoted
`name_shares_first_word`	rejected

the Judge contradicted it on “Honeywell Intl”

5 of 10 held-out suppliers settled with **no model call**; accuracy stayed **13/13**

Built to run from a clean clone

The system

Language: TypeScript 6 (strict), Node.js, no framework

API: `POST /run` on port 8000

Analyst page: its own port, 8001; HTML, CSS and type-checked JavaScript

Data: Zetaris Lightning SQL over its REST API

AI: Meterless Nightshift → NVIDIA `Nemotron 3 Ultra`

Reliability

Docker: one command; runs as a non-root user

Sample mode: replays recorded Zetaris and Nightshift answers, no network or keys

Traces: one JSONL file per run: every agent step, query and model

Tests: 178 unit tests, both official Meterless conformance suites, strict lint, mutation testing, a secrets scan

Failure handling: timeouts, retries with backoff; one failed supplier never stops the run

Sponsor technology doing the real work

Zetaris

The only data layer

- The ERP (Postgres) registered as a JDBC source
- The SIC list as a cached REST table and view
- The agents register GLEIF and EDGAR records as REST tables at runtime
- One federated query per supplier, with the learned rules as columns

Meterless engines

Both pass the official tests

- **World Model:** the Resolver's entity matching and scoring
- **H-MEM:** the Rule Smith's memory, dreaming and trust ledger
- **Conformance:** H-MEM 13/13, World Model 8/8

Nightshift and NVIDIA

- Nightshift is the only AI provider (OpenAI-format API)
- `Nemotron 3 Ultra` gives every verdict
- Each request is logged with the model that answered

What’s new in how we used the sponsors

1 Judgments become governed SQL

the model’s proven verdicts turn into rules Zetaris enforces

2 Proven before trusted

back-tested against every verdict; a wrong rule is visibly rejected; a rule later overturned is demoted

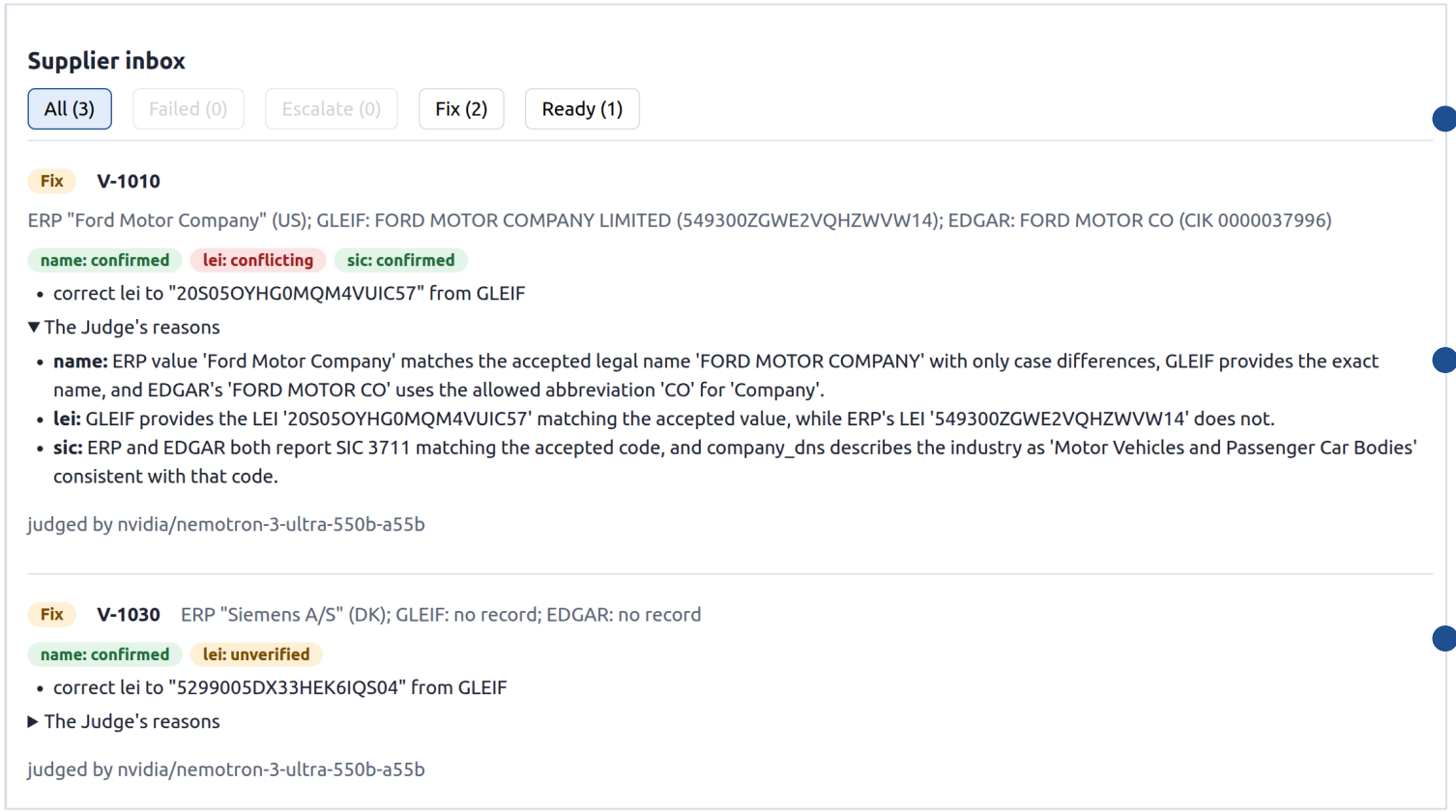
3 Measured, not claimed

on held-out suppliers, half the model calls saved, accuracy unchanged

4 Zetaris grows during the run

the agents add the registry sources they need, and Zetaris evaluates what they learn

The analyst’s view



1. **Run summary:** live mode, decisions, score against the answer key
2. **Supplier inbox:** filter by Escalate, Fix or Ready; each card shows the facts, the correction, the Judge’s reasons and the model
3. **Rules and trace:** what the agents learned, and every step, filterable by supplier

Works from the keyboard alone and on a phone-sized screen.

Every step is in the trace

Rules the Rule Smith learned

Proposed from the Judge's verdicts on the learning batch; promoted only when the Judge confirmed every supplier the rule held for. Promoted rules settle facts in Zetaris SQL without a model call.

Rule	Status	Support	Why
name_is_gleif_legal_name The ERP name is GLEIF's legal name, ignoring case, spaces and punctuation ▶ SQL	promoted	7	held for 7 learning verdicts and the Judge confirmed every one
name_shares_first_word The ERP name starts with the same word as GLEIF's legal name ▼ SQL `split(upper(e.vendor_name), ' ')[0] = split(upper(g.data.attributes.entity.legalName.name), ' ')[0]`	rejected	8	the Judge contradicted it on V-1019
lei_is_claimed_record The ERP's LEI is the LEI of the GLEIF record it names ▶ SQL	promoted	10	held for 10 learning verdicts and the Judge confirmed every one
sic_is_listed_edgar_code The ERP's SIC code is EDGAR's, and the company_dns list has it ▶ SQL	promoted	7	held for 7 learning verdicts and the Judge confirmed every one
sic_missing_from_list EDGAR's SIC code is missing from the company_dns list, so the industry is disputed ▶ SQL	promoted	3	held for 3 learning verdicts and the Judge confirmed every one

Learned rules: promoted and rejected

Trace

Supplier All logs/trace-deck-ford.jsonl

Time	Supplier	Agent	Action	Result	Status
+0.0 s	run	Scout	zetaris_query	▶ 31 rows	success
+0.0 s	run	Pipeline → Scout	start	1 of 31 ERP suppliers, 2 at a time	success
+0.1 s	V-1010	Scout	zetaris_query	▶ 50 rows	success
+0.2 s	V-1010	Scout	zetaris_query	▶ 17 rows	success
+5.2 s	V-1010	Scout	zetaris_query	▶ 1 rows	success
+5.2 s	V-1010	Scout → Resolver	gather_evidence	ERP "Ford Motor Company" (US); GLEIF: FORD MOTOR COMPANY LIMITED (549300ZGWE2VQHZWVW14); EDGAR: FORD MOTOR CO (CIK 0000037996)	success
+5.3 s	V-1010	Resolver	zetaris_query	▶ 50 rows	success
+6.4 s	V-1010	Resolver	zetaris_query	▶ 2 rows	success
+6.4 s	V-1010	Resolver	world_model_score	▶ World Model scored 4 GLEIF entities: 1 at or above the review band; FORD MOTOR COMPANY 1.00 (auto-merge)	success
+6.4 s	V-1010	Resolver → Judge	resolve_identity	clear: FORD MOTOR COMPANY (20S05OYHG0MQM4VUIC57); claimed LEI 549300ZGWE2VQHZWVW14 is FORD MOTOR COMPANY LIMITED in GB	success
+26.5 s	V-1010	Judge	nightshift_chat nvidia/nemotron-3-ultra-550b-a55b	▶ 2187 characters in 20.1 s	success
+26.5 s	V-1010	Judge → Decision	judge_facts nvidia/nemotron-3-ultra-550b-a55b	name: ERP value 'Ford Motor Company' matches the accepted legal name 'FORD MOTOR COMPANY' with only case differences, GLEIF provides the exact name, and EDGAR's 'FORD MOTOR CO' uses the allowed abbreviation 'CO' for 'Company'. lei: GLEIF provides the LEI '20S05OYHG0MQM4VUIC57' matching the accepted value, while ERP's LEI '549300ZGWE2VQHZWVW14' does not. sic: ERP and EDGAR both report SIC 3711 matching the accepted code, and company_dns describes the industry as 'Motor Vehicles and Passenger Car Bodies' consistent with that code.	success
+26.5 s	V-1010	Decision	decide	fix; correct lei to "20S05OYHG0MQM4VUIC57" from GLEIF	success

Trace for one supplier, step by step

The Zetaris queries, the World Model's scores, each Nightshift request with the model that answered, and each decision.

Future enhancements

Next

- Keep learned rules across runs, so each request starts with what the last one proved
- Register promoted rules as Zetaris data-quality rules
- Let the model propose new rule SQL, gated by the same back-test
- Write accepted corrections back to the ERP, with the analyst's approval
- More checks: addresses, ownership, sanctions screening, non-US industry codes

Honest limits today

- Five hand-written rule shapes
- Learning happens within one run
- A small, self-labelled test set of 31 suppliers
- `Nemotron 3 Ultra` takes 5 to 60 s per verdict

Provenance

Fewer model calls as it learns.

Every rule proven first.

Every step traced.

github.com/Vitalisn4/provenance

Track 1, Solving Fragmented Intelligence · Built with Zetaris, Meterless, NVIDIA Nemotron and Cursor